

Trabajo Práctico Integrador

Gestión de Datos de Países en Python: filtros, ordenamientos y estadísticas

Institución: Universidad Tecnológica Nacional (UTN)

Carrera: Tecnicatura Universitaria en Programación a Distancia

Materia: Programación 1

Integrantes: Matias Fernando Nuñez
Rodrigo Nahuel Guggiana

Fecha de entrega: 15 de junio de 2026

Índice

1. Introducción
2. Marco Teórico
 - 2.1. Listas
 - 2.2. Diccionarios
 - 2.3. Funciones y modularización
 - 2.4. Estructuras condicionales y repetitivas
 - 2.5. Ordenamientos
 - 2.6. Estadísticas básicas
 - 2.7. Archivos CSV
 - 2.8. Manejo de errores con try/except
3. Decisiones Técnicas y Arquitectura
 - 3.1. Diseño en módulos
 - 3.2. Estructura de datos elegida
 - 3.3. Diagrama de flujo de operaciones
 - 3.4. Validaciones implementadas
4. Capturas de ejecución
5. Dificultades y Conclusiones
6. Bibliografía y Webgrafía
7. Enlaces al repositorio y al video

1. Introducción

El presente Trabajo Práctico Integrador (TPI) consolida los conceptos estudiados a lo largo de la materia **Programación 1** mediante el desarrollo de una aplicación Python que **gestiona información sobre países** a partir de un archivo CSV.

El sistema permite realizar las siguientes operaciones a través de un menú interactivo:

- Cargar y persistir un catálogo de países en formato CSV.
- Agregar y actualizar países.
- Buscar por nombre con coincidencia parcial o exacta.
- Filtrar por continente o por rangos de población y superficie.
- Ordenar por nombre, población o superficie (ascendente o descendente).
- Generar estadísticas: país con mayor y menor población, promedios, distribución por continente.

El dominio elegido — países del mundo — permite trabajar con un **dataset de 120 registros reales** verificables, lo que da sentido a las estadísticas y operaciones de filtrado.

2. Marco Teórico

2.1. Listas

Las **listas** son estructuras de datos **mutables, ordenadas y heterogéneas** que permiten almacenar una colección de elementos. En Python se construyen con corchetes `[ ]` y se indexan desde `0`.

Operación	Sintaxis	Costo
Acceso por índice	<code>lista[i]</code>	$O(1)$
Recorrer	<code>for elem in lista:</code>	$O(n)$
Agregar al final	<code>lista.append(x)</code>	$O(1)$
Filtrar	<code>[x for x in lista if condición]</code>	$O(n)$
Ordenar	<code>sorted(lista, key=..., reverse=...)</code>	$O(n \log n)$

Las listas son la **estructura de almacenamiento principal** del sistema: el catálogo completo de países vive en una sola lista en memoria.

Fuente: Python Software Foundation, **Data Structures — Python 3 docs** (<https://docs.python.org/3/tutorial/datastructures.html>).

2.2. Diccionarios

Los **diccionarios** son colecciones de pares **clave-valor**, donde las claves son únicas y permiten acceso en tiempo constante $O(1)$ promedio. Se construyen con llaves `{ }`.

En este proyecto, **cada país es un diccionario** con exactamente cuatro claves:

```
{
    "nombre": "Argentina",
    "poblacion": 45376763,
    "superficie": 2780400,
    "continente": "América",
}
```

Las ventajas de usar diccionarios en lugar de listas posicionales son:

- **Legibilidad:** `pais["nombre"]` comunica más que `pais[0]`.
- **Mantenimiento:** si se agrega una columna al CSV, los accesos por clave siguen funcionando.
- **Auto-documentación:** las claves explican el modelo.

2.3. Funciones y modularización

Una **función** encapsula una operación bajo un nombre, recibe parámetros y opcionalmente devuelve un valor. En Python se definen con `def`.

El principio aplicado en este proyecto es **"una función = una responsabilidad"**: cada función hace una sola cosa y la hace bien. Esto se traduce en funciones puras cuando es posible (sin efectos colaterales) y mutaciones explícitas cuando no.

A nivel de archivo, la modularización se distribuyó en cuatro módulos:

Módulo	Responsabilidad
main.py	Menú interactivo y orquestación
países.py	CRUD + búsqueda + lectura/escritura CSV
filtros.py	Filtros + ordenamientos
estadisticas.py	Cálculos estadísticos

2.4. Estructuras condicionales y repetitivas

- **Condicionales** (`if`, `elif`, `else`): toman decisiones en función de una condición booleana.
- **Bucles** `while`: repiten un bloque mientras una condición sea verdadera. Útiles para insistir hasta validar input.
- **Bucles** `for`: iteran sobre un iterable (lista, archivo, range). Útiles para recorrer la lista de países.

2.5. Ordenamientos

Python provee la función built-in `sorted(iterable, key=..., reverse=...)` que devuelve una **nueva lista ordenada** sin modificar la original. Es una buena práctica, porque las funciones de ordenamiento no tienen efectos colaterales.

El parámetro `key` acepta una función que se aplica a cada elemento para extraer el valor por el cual ordenar. En este proyecto usamos **funciones lambda**:

```
sorted(países, key=lambda p: p["poblacion"], reverse=True)
```

El algoritmo interno de `sorted()` es **Timsort**, una variante híbrida de mergesort y insertion sort, con complejidad **$O(n \log n)$** en el peor caso.

2.6. Estadísticas básicas

Función	Implementación
Máximo / mínimo	<code>max(países, key=lambda p: p['poblacion'])</code>
Promedio	<code>sum(p['poblacion'] for p in países) / len(países)</code>
Conteo por categoría	<code>dict.get(clave, 0) + 1</code> sobre cada elemento

2.7. Archivos CSV

CSV (*Comma-Separated Values*) es un formato de texto plano para datos tabulares. Python provee el módulo `csv` de la biblioteca estándar, con dos clases clave: `csv.DictReader` (lee filas como diccionarios) y `csv.DictWriter` (escribe diccionarios).

Parámetro de <code>open()</code>	Por qué
<code>encoding='utf-8'</code>	Soporta tildes, eñes, caracteres especiales.
<code>newline=""</code>	Crítico en Windows al escribir CSV: sin esto aparecen filas vacías.

2.8. Manejo de errores con try/except

El bloque try/except captura excepciones en runtime para evitar que el programa termine abruptamente. En este proyecto se usa en cuatro contextos:

- **Captura de entradas no numéricas:** `int(input(...))` lanza `ValueError`.
- **Errores de archivos:** `FileNotFoundError` o `ValueError` de formato.
- **Reglas de negocio con `raise ValueError`:** duplicados, rangos invertidos, valores negativos.
- **Opciones de menú inválidas:** opciones no numéricas o fuera de rango.

3. Decisiones Técnicas y Arquitectura

3.1. Diseño en módulos

Se eligió separar el código en cuatro archivos según la responsabilidad de cada uno. Esta es una práctica estándar en proyectos Python: **bajo acoplamiento, alta cohesión**.

```
codigo/  
├─ main.py           # Punto de entrada (orquesta el menú)  
├─ paises.py         # CRUD + búsqueda + I/O del CSV  
├─ filtros.py        # Filtros + ordenamientos (puros)  
├─ estadisticas.py   # Estadísticas (puras)  
└─ paises.csv        # Dataset
```

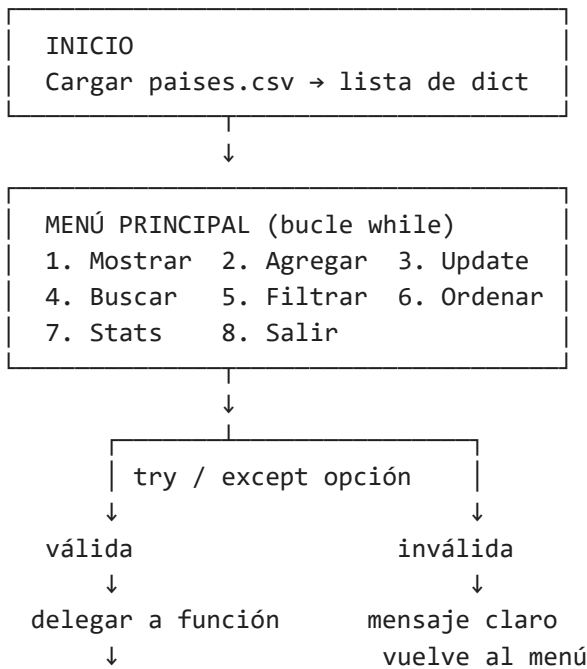
3.2. Estructura de datos elegida

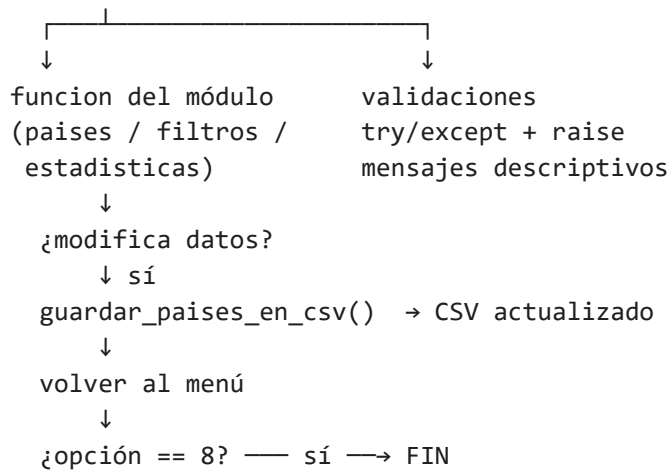
La estructura central es una **lista de diccionarios**: `paises[]`, donde cada elemento es un dict con cuatro claves.

Por qué lista de diccionarios y no otras alternativas:

Alternativa	Por qué se descartó
Listas paralelas	Si una cambia de orden, todo se desincroniza.
Tuplas posicionales	Menor legibilidad: <code>pais[2]</code> no comunica 'superficie'.
Clases	La consigna no las requiere. Diccionarios alcanzan.
Dict indexado por nombre	Más rápido para buscar pero complica filtros y orden.

3.3. Diagrama de flujo de operaciones





3.4. Validaciones implementadas

Funcionalidad	Validaciones
Carga del CSV	Archivo existe + columnas correctas + tipos convertibles
Agregar país	Nombre no vacío + no duplicado + población y superficie enteros positivos
Actualizar país	País existe + nuevos valores son enteros positivos
Buscar	Texto no vacío + tipo (1 o 2) válido
Filtrar por rango	Mínimo ≤ máximo, ambos no negativos
Filtrar por continente	Texto no vacío (case-insensitive)
Ordenar	Criterio (1-3) y sentido (1-2) válidos
Menú principal	Opción entero entre 1 y 8

4. Capturas de ejecución

A continuación se muestran **capturas de pantalla de la ejecución real** del programa en la terminal, abarcando el flujo normal y casos de error controlados. Las imágenes se generan automáticamente a partir de la salida real del programa con `generar_capturas.py`.

4.1. Inicio del programa (carga del CSV y menú)



```
Windows PowerShell - python main.py

[OK] Se cargaron 120 países desde 'países.csv'.

=====
GESTIÓN DE PAÍSES - Trabajo Práctico Integrador
=====
1. Mostrar todos los países
2. Agregar país
3. Actualizar país (población / superficie)
4. Buscar país por nombre
5. Filtrar países (continente / rango)
6. Ordenar países
7. Mostrar estadísticas
8. Salir
```

Figura 1. Al iniciar, el sistema carga los 120 países desde `países.csv` y muestra el menú principal.

4.2. Estadísticas (opción 7)

```
Windows PowerShell - python main.py

[OK] Se cargaron 120 países desde 'países.csv'.

=====
GESTIÓN DE PAÍSES - Trabajo Práctico Integrador
=====

1. Mostrar todos los países
2. Agregar país
3. Actualizar país (población / superficie)
4. Buscar país por nombre
5. Filtrar países (continente / rango)
6. Ordenar países
7. Mostrar estadísticas
8. Salir

=====
Seleccione una opción (1-8): 7

=== Estadísticas del dataset ===
Total de países cargados: 120

País con MAYOR población: China (1,439,323,776 habitantes)
País con MENOR población: Samoa (198,414 habitantes)

Promedio de población :      62,087,051.16 habitantes
Promedio de superficie:      1,019,771.54 km²

Cantidad de países por continente:
Asia      : 36 países
Europa    : 34 países
América    : 22 países
África     : 22 países
Oceanía    : 6 países
```

Figura 2. Estadísticas del dataset: país con mayor y menor población, promedios y conteo por continente.

4.3. Búsqueda parcial "ar" (opción 4)

```
Windows PowerShell - python main.py

[OK] Se cargaron 120 países desde 'países.csv'.

=====
GESTIÓN DE PAÍSES - Trabajo Práctico Integrador
=====

1. Mostrar todos los países
2. Agregar país
3. Actualizar país (población / superficie)
4. Buscar país por nombre
5. Filtrar países (continente / rango)
6. Ordenar países
7. Mostrar estadísticas
8. Salir

=====
Seleccione una opción (1-8): 4

=== Buscar país por nombre ===
Texto a buscar: ar
Tipo de búsqueda: 1) Exacta  2) Parcial (contiene el texto)
Seleccione (1 o 2): 2

=== Resultados para 'ar' (11 resultado/s) ===
#      Nombre                Población  Superficie km²  Continente
-----
1      Argentina               45,376,763    2,780,400  América
2      Nicaragua                 6,624,554    130,373  América
3      Paraguay                  7,132,538    406,752  América
4      Bulgaria                   6,927,288    110,879  Europa
5      Dinamarca                  5,831,404     42,933  Europa
6      Arabia Saudita            34,813,871    2,149,690  Asia
7      Myanmar                   54,409,800    676,578  Asia
8      Qatar                      2,881,053     11,586  Asia
9      Argelia                   43,851,044    2,381,741  África
10     Madagascar                27,691,018     587,041  África
11     Marruecos                  36,910,560     446,550  África
-----
```

Figura 3. Búsqueda parcial case-insensitive: devuelve los países cuyo nombre contiene "ar".

4.4. Filtro por rango inválido (opción 5) — manejo de error

```
Windows PowerShell - python main.py

[OK] Se cargaron 120 países desde 'países.csv'.

=====
GESTIÓN DE PAÍSES - Trabajo Práctico Integrador
=====
1. Mostrar todos los países
2. Agregar país
3. Actualizar país (población / superficie)
4. Buscar país por nombre
5. Filtrar países (continente / rango)
6. Ordenar países
7. Mostrar estadísticas
8. Salir

=====
Seleccione una opción (1-8): 5

=== Filtrar países ===
1. Por continente
2. Por rango de población
3. Por rango de superficie
0. Volver
Seleccione: 2
Población mínima: 1000000
Población máxima: 100
Error: El mínimo (1,000,000) no puede ser mayor que el máximo (100).
```

Figura 4. Ante un rango invertido (mínimo > máximo), el sistema informa el error y no se interrumpe.

4.5. Agregar país duplicado (opción 2) — manejo de error

```
Windows PowerShell - python main.py

[OK] Se cargaron 120 países desde 'países.csv'.

=====
GESTIÓN DE PAÍSES - Trabajo Práctico Integrador
=====

1. Mostrar todos los países
2. Agregar país
3. Actualizar país (población / superficie)
4. Buscar país por nombre
5. Filtrar países (continente / rango)
6. Ordenar países
7. Mostrar estadísticas
8. Salir

=====
Seleccione una opción (1-8): 2

=== Agregar nuevo país ===
Nombre: Argentina
Continente: América
Población (entero > 0): 50000000
Superficie en km² (entero > 0): 2780400
Error: El país 'Argentina' ya existe.. No se agregó el país.
```

Figura 5. El alta valida duplicados (case-insensitive) y rechaza el país con un mensaje claro.

5. Dificultades y Conclusiones

Dificultades enfrentadas

- 1. Persistencia en CSV (Windows + UTF-8).** La escritura del CSV en Windows requería el parámetro `newline=""` en `open()` — sin él aparecían filas vacías intercaladas. Lo resolvimos consultando la documentación oficial del módulo `csv`.
- 2. Búsqueda case-insensitive con tildes.** Decidir si "Mexico" debe coincidir con "México" no era trivial. Optamos por hacer comparaciones con `.strip().lower()` que ignora capitalización pero respeta las tildes.
- 3. Diseño modular.** Empezamos con todo en un solo archivo, pero el código se volvía difícil de leer. Refactorizamos en 4 módulos por responsabilidad. La regla "una función = una responsabilidad" guio toda la división.
- 4. Validaciones de filtros con rangos.** Tuvimos que decidir qué hacer si el usuario ingresa un mínimo mayor que el máximo. Optamos por lanzar `ValueError` con mensaje descriptivo en lugar de devolver lista vacía silenciosamente.

Conclusiones grupales

El TPI nos permitió **integrar todos los conceptos de la materia** en un sistema funcional: listas y diccionarios como estructura central, funciones modularizadas en cuatro archivos, bucles para recorrer y persistir el menú, condicionales para validaciones, `try/except` y `raise` para manejo robusto de errores, archivos CSV para persistencia, y estadísticas básicas con `max`, `min`, promedios y conteos.

Lo más valioso fue ver cómo **la modularización mejora la legibilidad y la mantenibilidad**: cada función hace una sola cosa, cada módulo tiene una responsabilidad clara, y la lógica central (`main.py`) solo orquesta. Esta arquitectura escala bien.

Otro aprendizaje clave fue la importancia de **validar todas las entradas del usuario y dar mensajes de error claros**. Un programa robusto no es solo el que funciona en el caso feliz: es el que nunca crashea y comunica los problemas con claridad.

6. Bibliografía y Webgrafía

- Python Software Foundation. *The Python Standard Library*. <https://docs.python.org/3/library/index.html>
- Python Software Foundation. *Module csv*. <https://docs.python.org/3/library/csv.html>
- Python Software Foundation. *Errors and Exceptions*. <https://docs.python.org/3/tutorial/errors.html>
- Python Software Foundation. *Sorting HOW TO*. <https://docs.python.org/3/howto/sorting.html>
- Python Software Foundation. *Data Structures*. <https://docs.python.org/3/tutorial/datastructures.html>
- van Rossum, G. et al. *PEP 8 — Style Guide for Python Code*. <https://peps.python.org/pep-0008/>
- van Rossum, G. *PEP 257 — Docstring Conventions*. <https://peps.python.org/pep-0257/>
- Cátedra UTN — Programación 1, 2026. Material de las unidades 1 a 10.

Datos del CSV: poblaciones y superficies obtenidas de Wikipedia (cifras 2020-2021).

7. Enlaces

- **Repositorio del proyecto:** <https://github.com/matiasfnunezdev/tpi-gestion-paises-utn>
- **Video demostrativo (YouTube):** <https://www.youtube.com/watch?v=jEk6lpOXb-Q>

Este documento se incluye dentro del ZIP de entrega y también está en docs/documentacion.pdf dentro del repositorio.